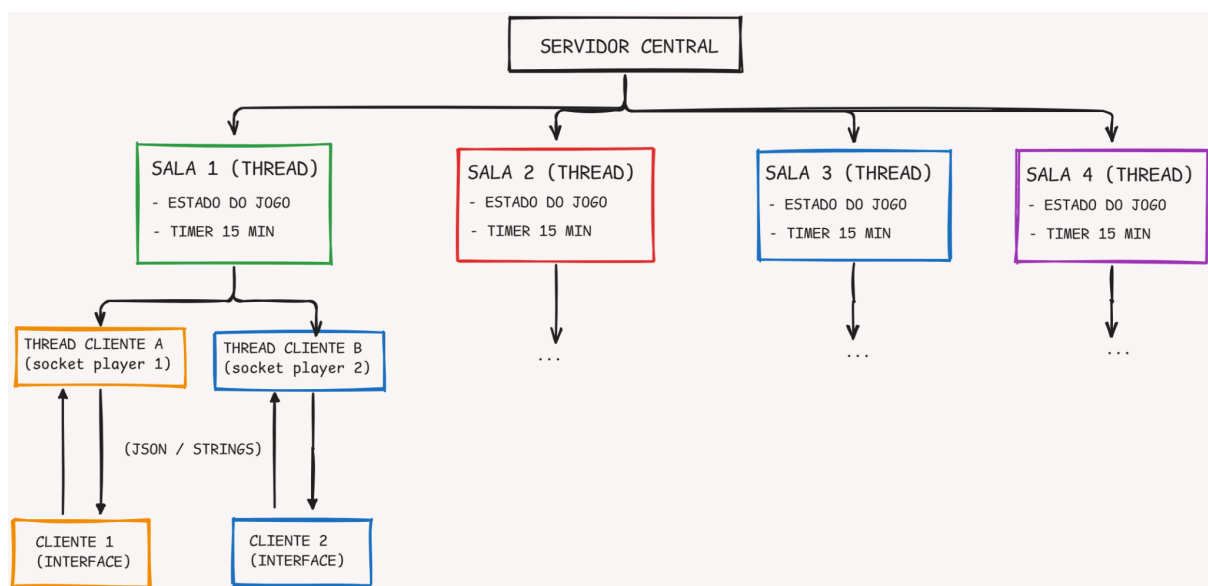


Projeto de Linguagem de Programação II

Objetivo Principal: produzir um jogo, clone de uno, aplicando todos os conceitos da disciplina de sincronização de threads, comunicação via sockets, consistência de estado e gerência de tempo.

Diagrama do Projeto (Fluxo de Comunicação):



Como o projeto envolve múltiplos clientes e múltiplas salas, a melhor abordagem é uma arquitetura **Servidor Multithreaded Centralizado**.

Passo a Passo estruturado do funcionamento

1. Inicialização do Servidor

- O **Server** inicia em uma porta específica (ex: **8080**) e inicializa uma lista estática/concorrente para gerenciar as 4 salas (ex: um **Map<String, Room>**).
- O servidor entra em um loop infinito (**while(true)**) com **serverSocket.accept()**, aguardando novas conexões de jogadores.

- Para cada nova conexão, o servidor cria uma Thread de Conexão específica para aquele cliente para ouvir suas requisições sem travar o servidor principal.

2. Handshake e Entrada na Sala (Lobby)

- O Cliente se conecta e envia seu nome e a ação desejada (Criar Sala ou Entrar na Sala com Código).
- **Tratamento Concorrente:** O servidor verifica a validade do código. Se aceito, associa a Thread daquele cliente à **Room** (Sala) correspondente.
- O estado da sala é atualizado (Lista de jogadores conectados) e um broadcast é enviado para todos na sala atualizarem suas telas de lobby.

3. Ciclo de Jogo (Início da Partida)

- Quando o criador da sala clica em "Iniciar" (ou a sala atinge o limite), a **Room** (que roda em sua própria Thread) dispara o loop do jogo:
 1. Instância o Baralho (embaralhado de forma segura).
 2. Distribui 7 cartas para cada jogador (enviando as mensagens TCP customizadas para cada socket).
 3. Define o primeiro jogador e inicia o **Timer Global da Partida (15 min)** e o **Timer do Turno (15 s)**.

4. O Turno e Sincronização (O Coração de PCD)

- O servidor envia uma mensagem para o jogador da vez: **SEU_TURNO**. Para os demais, envia **TURNO_DE: [Nome]**.
- **Gerência de Tempo:** A Thread da Sala inicia uma contagem regressiva de 15 segundos. Se o jogador responder via socket dentro do tempo, a jogada é validada. Se o tempo esgotar, o servidor força o jogador a comprar uma carta e passa a vez.
- **Validação de Estado:** O servidor é o dono da verdade. O cliente tenta jogar uma carta; o servidor valida se a cor ou o número

batem com o topo do descarte (a fila que você mencionou). Se sim, atualiza o topo e passa o turno.

5. Mecânica do "UNO" e Fim de Jogo

- **Botão de UNO:** Quando o jogador joga sua penúltima carta, ele tem um curto espaço de tempo (ou deve fazer isso simultaneamente à jogada) para clicar em "UNO". Se outro jogador clicar em "DORMIU!" (ou o servidor detectar a falta do comando) antes da vez passar, o jogador recebe uma punição (compra +2 cartas).
- **Vitória:** Quando a lista de cartas de um jogador zera, o servidor encerra os timers da sala, envia um broadcast `FIM_DE_JOGO;` `VENCEDOR: [Nome]` acompanhado do estado final das mãos de todos os jogadores para exibição na interface.

Requisitos interessantes para se incluir:

1. Tratamento de Quedas de Conexão (Fault Tolerance)

- O que acontece se o socket de um jogador fechar abruptamente no meio da partida? *

Solução: Adicionar um timeout de leitura (`SO_TIMEOUT`) ou tratar o `IOException`. Se um jogador cair, o servidor pode removê-lo da partida, transformar o turno dele em "passou a vez automaticamente" ou transformá-lo em um Bot simples para não estragar o jogo dos outros 7.

2. Protocolo de Comunicação Estruturado (JSON ou String Tokenizada)

- Definir mensagens claras trafegando no Socket facilita muito a depuração.

- *Exemplo String Tokenizada:*
`SALA_JOIN;codigo123;NomeDoPlayer`
- *Exemplo JSON:* ``json { "action": "PLAY_CARD", "card": { "color": "RED", "value": 7 } }

3. Sincronização Estrita no Baralho e no Descarte

- Usar estruturas de dados thread-safe do Java (como `Collections.synchronizedList` ou `CopyOnWriteArrayList`) para o baralho e para a lista de jogadores da sala, evitando `ConcurrentModificationException` quando um jogador tenta comprar cartas ao mesmo tempo que o timer do turno decide puni-lo por tempo esgotado.

Escolha da Interface Gráfica (JavaSwing)

Embora o JavaFX seja mais moderno e permita interfaces mais bonitas (estilizadas com CSS), ele exige uma curva de aprendizado maior, configurações extras de ambiente (módulos do Java) e ferramentas externas como o Scene Builder. O **Swing já vem embutido no Java (JDK)**, não precisa configurar absolutamente nada no projeto e resolve o problema imediatamente.

FASES DE IMPLEMENTAÇÃO

Fase 1: O Domínio do Jogo (A Base Lógica) Antes de pensar em rede ou threads, a lógica do UNO precisa existir e ser testável localmente.

- **Enum CardColor e Enum CardValue:** Para representar as cores (RED, BLUE, GREEN, YELLOW, BLACK) e os valores/ações (0-9, SKIP, REVERSE, DRAW_TWO, WILD, WILD_DRAW_FOUR).
- **class Card:** Objeto imutável contendo cor e valor.

- **class Deck:** Responsável por gerar as 108 cartas, embaralhar (`Collections.shuffle`) e fornecer métodos como **`drawCard()`**. Precisa ser thread-safe na fase de rede.
- **class PlayerState:** Representa o jogador apenas no nível da lógica (nome, lista de cartas na mão).

Fase 2: Infraestrutura de Rede e Handshake (Sockets Básicos)

Estabelecer a comunicação cliente-servidor sem ainda se preocupar com as regras do jogo.

- **class ServerMain:** A classe que possui o `ServerSocket`. Ela escuta a porta e aceita conexões em um loop **`while(true)`**.
- **class ClientConnection implements Runnable:** A Thread dedicada para cada cliente conectado. Ela contém o **`Socket`, `ObjectInputStream` e `ObjectOutputStream` (ou `BufferedReader/PrintWriter` se usar **JSON/String**).**
- **class ProtocolMessage:** Estrutura padrão de envio de mensagens (ex: JSON ou uma string delimitada por ponto e vírgula).

Fase 3: Gerenciamento de Salas e Concorrência Integração dos clientes em grupos fechados e garantia de que o estado não seja corrompido por acessos simultâneos.

- **class RoomManager:** Singleton ou classe estática que detém o mapa de salas ativas (**ex: `ConcurrentHashMap<String, Room>`**).
- **class Room implements Runnable:**

Cada sala é uma Thread que gerencia sua própria partida. Ela possui:

- **O Deck da sala.**
- A lista de **`ClientConnection`** dos jogadores daquela sala (utilize **`CopyOnWriteArrayList`** para manter o estado thread-safe).
- **A pilha de descarte.**

- O índice de quem é a vez e o sentido do jogo.

Fase 4: Timers e Consistência (O Desafio Principal) Implementar a passagem de tempo sem bloquear a escuta de mensagens do servidor.

- **class TurnTimer:** Utilize um **ScheduledExecutorService** ou **java.util.Timer** dentro da Room.
- **Cancelamento Concorrente:** Se o jogador enviar uma jogada válida via socket *antes* dos 15 segundos, a thread da **Room** deve cancelar essa tarefa agendada e iniciar um novo timer para o próximo jogador.
- **Validação Centralizada:** Crie um método boolean **isValidPlay(Card playedCard, Card topDiscard)** na Room. O cliente *nunca* decide se a jogada foi válida, ele apenas pede para jogar e o servidor aprova ou recusa.

Fase 5: Interface Gráfica (Swing) do Cliente Deixe a UI para o final. O cliente deve rodar como um ouvinte passivo do servidor.

- **class ClientMain:** Conecta no socket e abre o Menu/Lobby.
- **class GameWindow extends JFrame:** A tela do jogo. Renderiza as cartas da mão e o topo do descarte baseado apenas nas informações que o servidor envia.
- **class ServerListener implements Runnable:** Uma thread rodando no cliente focada apenas em escutar os broadcasts do servidor (ex: "Atualize sua mão", "Turno do João") e atualizar o Swing utilizando **SwingUtilities.invokeLater()**.